AE 370 Group Project 1

Maria Dalton
Morgan Waller
Yogeshwari Kant
Justin Chiang

Group 18
Aerospace Engineering, University of Illinois Urbana-Champaign

Oct 28, 2026

1 The premise

1.1 Framing the Question

The objective of this project is to investigate how rocket motor burn duration and propellant mass fraction affect the maximum altitude achieved by a vertically launched sounding rocket. Its maximum altitude depends on factors such as motor burn duration, propellant mass, gravitational forces, and aerodynamic drag. Increasing propellant mass changes the rocket's initial mass and mass depletion rate; whether it also increases impulse depends on the specified thrust and burn duration. Additionally, higher ascent velocities can increase aerodynamic drag and therefore reduce the altitude achieved. The engineering question we aim to investigate is: How do rocket motor burn duration and propellant mass fraction influence the maximum apogee altitude of a vertically launched sounding rocket, and how much does atmospheric drag reduce apogee for each motor configuration? For the parameter studies, thrust T_0, dry mass, geometry, and environmental parameters are held fixed at their Table 1 values. Define the propellant mass fraction by

$$\phi = \frac{m_p}{m_0}, \qquad m_0 = \frac{m_{\text{dry}}}{1-\phi}, \qquad m_p = m_0 - m_{\text{dry}}, \qquad \dot{m} = \frac{m_p}{t_{\text{burn}}}.$$

Varying burn duration at fixed thrust changes total impulse $I = T_0 t_{\text{burn}}$. Varying ϕ at fixed thrust and burn duration changes mass, but not impulse. These are independent model-sensitivity studies, not comparisons of motors with a fixed specific impulse. The implied effective exhaust velocity $T_0/\dot{m}$ can vary between configurations. The drag penalty is defined by $h_{\text{ap}}(C_d = 0) - h_{\text{ap}}(C_d = 0.45)$ for otherwise identical parameters. Only immediate-lift-off configurations with $T_0 > m_0 g$ are considered; ground contact and delayed lift-off are not modeled.

1.2 Relationship with Initial Value Problem

In order to accurately predict the flight trajectory of the sounding rocket, multiple parameters must be considered. The acceleration of the rocket would be dependent on the following parameters:

$$\text{Signed aerodynamic resistance: } D(v,h) = \frac{1}{2}\rho(h)\, v|v| C_d A$$

$$\text{Thrust: } T(t) = \begin{cases} T_0 & \text{if } 0 \le t < t_{burn} \\ 0 & \text{if } t \ge t_{burn} \end{cases}$$

$$\text{Mass: } m(t) = \begin{cases} m_0 - \dot{m}t & \text{if } 0 \le t < t_{burn} \\ m_{\text{dry}} & \text{if } t \ge t_{burn} \end{cases}$$

We will need to incorporate atmospheric drag into our problem by using the following equations: Atmospheric density equation:

$$\rho(h) = \rho_0 e^{-h/H}$$

Substituting atmospheric density into drag:

$$D(v,h) = \frac{1}{2}\left(\rho_0 e^{-h/H}\right) v|v| C_d A$$

1.2.1 Equations Combined

$$\boxed{\frac{dh}{dt} = v}$$

$$\boxed{\frac{dv}{dt} = \frac{T(t)}{m(t)} - g - \frac{\rho_0 C_d A}{2m(t)} \cdot e^{-h/H} v|v|}$$

1.2.2 Initial Conditions

$$h(0) = 0,\ v(0) = 0,\ m(0) = m_0$$

An Initial Value Problem (IVP) is necessary for this project because it allows us to predict the rocket's altitude and velocity over time using known starting conditions. The rocket's motion is constantly changing due to thrust, decreasing mass, gravity, and atmospheric drag. Since these factors make the governing differential equations difficult to solve analytically, we use a numerical method such as fourth-order Runge-Kutta (RK4) to approximate the rocket's trajectory. By solving the IVP, we can determine the rocket's maximum altitude (apogee) and analyze how burn duration and propellant mass affect its performance.

1.3 Reason for Numerical Methods

From the governing equations, it is evident that the terms are highly coupled, non-linear, and piecewise. The acceleration, $\frac{dv}{dt}$, depends on the drag, thrust, and mass of the rocket. Because velocity determines the altitude h, and altitude dictates the atmospheric air density, the drag force relies simultaneously on the exact integral of velocity (altitude) and the square of the velocity itself. These dependencies couple the two state equations. No closed-form solution for the full model is established here; numerical integration provides a practical way to evaluate its trajectories across the parameter study. Furthermore, the mass of the rocket decreases dynamically as propellant is consumed, and the thrust acts as a piecewise function that abruptly drops to zero at burnout.

We therefore use numerical methods to simulate the full trajectory, reserving analytically solvable special cases for verification. By defining the starting state, an Initial Value Problem is established. A numerical solver uses these initial conditions to evaluate acceleration at successive stage states. This iteratively approximates the instantaneous velocity, altitude, and drag as the rocket ascends.

1.4 Mathematical Functions and Parameters

The vertical flight of the sounding rocket is formulated as a second-order initial value problem expressed as a coupled system of first-order ordinary differential equations in state-space form:

$$\mathbf{y}(t) = \begin{bmatrix} h(t) \\ v(t) \end{bmatrix}, \quad \frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}) = \begin{bmatrix} f_1(t, h, v) \\ f_2(t, h, v) \end{bmatrix} \tag{1}$$

where $h(t)$ is the vertical altitude and $v(t)$ is the vertical flight velocity.

1.5 Auxiliary Models and Assumptions

Thrust already represents the propulsive momentum flux. Thus the variable-mass equation is $m\dot{v} = T - D - mg$; adding a separate $-\dot{m}_{\text{state}} v$ force would double-count the mass-flow effect in this thrust convention. During ascent $D \geq 0$, so $-D$ opposes upward motion. The exponential atmosphere approximates an isothermal atmosphere with constant gravity. The prescribed positive consumption rate is $\dot{m} = (m_0 - m_{\text{dry}})/t_{\text{burn}}$.

The numerical implementation augments the state to $u = [h, v, m]^T$, with $\dot{m}_{\text{state}} = -\dot{m}$ during powered flight and zero during coast. Here $\dot{m} > 0$ denotes the prescribed propellant consumption rate, not the signed derivative of the mass state. Integrating this third equation gives the same piecewise $m(t)$ used above.

The model assumes vertical point-mass motion, no wind or lift, constant gravity, a constant drag coefficient and reference area, and constant thrust within the burn. The parameters are illustrative model inputs rather than calibrated measurements of a particular motor. Constant C_d does not resolve changes with Mach number; numerical convergence therefore verifies the discretization, not the real-world fidelity of this simplified model.

1.5.1 Problem Parameters

The physical constants, vehicle parameters, and numerical parameters governing this investigation are summarized in Table 1.

Table 1: Physical, vehicle, and numerical parameters for the sounding rocket simulation.

Category	Parameter	Symbol	Value / Unit
Environment	Sea-level air density	ρ_0	1.225 kg/m^3
	Scale height	H	8500 m
	Standard acceleration of gravity	g	9.81 m/s^2
Rocket Geometry	Cross-sectional reference area	A	0.0314 m^2
	Aerodynamic drag coefficient	C_d	0.45
Mass & Propulsion	Initial gross mass	m_0	50.0 kg
	Dry mass	m_{dry}	20.0 kg
	Nominal thrust magnitude	T_0	2500 N
	Nominal burn duration	t_{burn}	10.0 s
Numerical Integration	Integration time step	Δt	Variable (s)
	Initial conditions	(h_0, v_0)	(0 m, 0 m/s)

2 The method

2.1 Justification

2.1.1 IVP Justification

The sounding rocket's ascent is modeled as an IVP because the trajectory can be determined from a known initial state and a system of differential equations. The initial conditions establish that the rocket will begin at ground level with zero vertical velocity. The equations of motion describe how the altitude and velocity will transform under the influence of thrust, gravity, and the aerodynamic drag. Propellant consumption changes rocket mass; gravity and aerodynamic drag enter the force balance. Because the governing equations are nonlinear and coupled, a numerical method is necessary to approximate the rocket's trajectory and determine its maximum altitude.

2.1.2 Justification for Fourth-order Runge-Kutta (RK4) method

A. Accuracy RK4 is a fourth-order numerical method, meaning that it provides relatively accurate approximations even when using moderately sized time steps. RK4 is a good choice because it achieves fourth order global accuracy. This means that its global numerical error behaves as:

$$\boxed{E_{\text{global}} = O(\Delta t^4)}$$

This means that as we make the time step smaller, the numerical error decreases very quickly. This is especially important when calculating apogee, since it occurs when the rocket's upward velocity reaches zero. RK4 provides a reliable way to predict this point without requiring extremely small time steps, as long as we handle engine burnout correctly and verify time-step convergence.

B. Stability The rocket's governing differential equations are:

$$\frac{dh}{dt} = v$$

$$\frac{dv}{dt} = \frac{T(t)}{m(t)} - g - \frac{\rho_0 C_d A}{2m(t)} e^{-h/H} v|v|$$

During each flight phase, the governing equations change smoothly as altitude, velocity, and mass change. For typical sounding rocket parameters, we expect this system to be non-stiff, meaning that its dynamics do not require extremely small time steps solely to maintain numerical stability within each smooth flight phase. Smoothness alone does not establish non-stiffness; the local stability and step-refinement checks below assess this choice. Because of this, an explicit method like RK4 is a reasonable choice.

C. Cost RK4 evaluates the differential equations four times during each step:

$$\boxed{\mathbf{y}_1,\ \mathbf{y}_2,\ \mathbf{y}_3,\ \mathbf{y}_4}$$

These four evaluations are combined to calculate the next altitude, velocity, and mass. By comparison, Euler's method only requires one evaluation per time step. At first, this might make RK4 seem more expensive. However, RK4 typically provides much higher accuracy at a given time step size, which can allow us to use fewer time steps to achieve a desired level of accuracy.

RK4 is an explicit method, meaning that it calculates the next state directly from the current state. It does not require matrix inversion or solving nonlinear algebraic systems during each step. For our relatively small, non-stiff system, this makes RK4 a practical computationally efficient choice.

2.2 The Mathematical Derivation and Properties

2.2.1 Taylor-Series Foundation

The Runge-Kutta 4th Order method is a numerical method that is meant to match the accuracy of a fourth-order Taylor series expansion without the calculation of complex higher order analytical derivatives. For an initial value problem that is defined as $\dfrac{du}{dt} = f(u,t)$, the Taylor series expansion of the state at the next time step $t_{k+1} = t_k + \Delta t$

$$u(t_{k+1}) = u(t_k) + \Delta t u'(t_k) + \frac{\Delta t^2}{2!}u''(t_k) + \frac{\Delta t^3}{3!}u'''(t_k) + \frac{\Delta t^4}{4!}u^{(4)}(t_k) + \mathcal{O}(\Delta t^5)$$

For a coupled, non-linear dynamical system, just like the problem discussed in this report, it is computationally expensive and complex deriving and evaluating these higher-order analytical derivatives u'', u''', u''''. The Runge-Kutta 4th Order method avoids these derivative evaluations by evaluating the first derivative of the function $f(u,t)$ at four stage states within each time step (t_k to t_{k+1}). Then, the Runge-Kutta 4th Order method would perfectly match the coefficient of the Taylor series up to the Δt^4 term by using the weighted average of the 4 evaluations at four stage states (three distinct times). This makes the truncation error at that interval $\mathcal{O}(\Delta t^5)$ every step, with a global truncation error of $\mathcal{O}(\Delta t^4)$ for the entire simulation.

2.2.2 Four Slopes

Following Lecture 10, the Runge-Kutta 4th Order method evaluates $u(t_{k+1})$ from $u(t_k)$ using a weighted average of the governing differential equation evaluated at four different locations within the time step (t_k to t_{k+1}). These evaluations, denoted as y_1, y_2, y_3, and y_4, represent the calculated slopes of the state trajectory at specific points.

The first slope y_1: The first slope or rate of change is calculated based on the governing differential equation f that defines how the system changes over time at the known location u_k at time $t = t_k$. Therefore, y_1 can be expressed as:

$$y_1 = f(u_k, t_k)$$

The second slope y_2: The second slope or rate of change is calculated at the location of the midpoint between t_k and t_{k+1}, which could be written as $t = t_k + \frac{1}{2}\Delta t$, based on the governing differential equation f. The estimated location at this midpoint used to calculate the rate of change is found by taking the known location u_k and projecting it forward by half a time step using the initial slope y_1. Therefore, y_2 can be expressed as:

$$y_2 = f\left(u_k + \frac{1}{2}\Delta t y_1, t_k + \frac{1}{2}\Delta t\right)$$

The third slope y_3: The third slope or rate of change is calculated at the exact same midpoint $t = t_k + \frac{1}{2}\Delta t$, based on the governing differential equation f. However, instead of projecting forward using the initial slope, this calculation estimates the midpoint state by projecting the known location u_k forward by half a time step using the newly calculated first midpoint slope y_2. Therefore, y_3 can be expressed as:

$$y_3 = f\left(u_k + \frac{1}{2}\Delta t y_2, t_k + \frac{1}{2}\Delta t\right)$$

The fourth slope y_4: The fourth slope or rate of change is calculated at the endpoint $t = t_k + \Delta t$, based on the governing differential equation f. The estimated location at this endpoint is determined by taking the known location u_k and projecting it forward a full time step Δt using the refined second midpoint slope y_3. Therefore, y_4 can be expressed as:

$$y_4 = f\left(u_k + \Delta t y_3, t_k + \Delta t\right)$$

The final equation: The final advanced location of the system is calculated by adding the product of the time step and the weighted average of the four previously evaluated slopes to the known location u_k. The specific weights are $\frac{1}{6}$, $\frac{2}{6}$, $\frac{2}{6}$, and $\frac{1}{6}$.

The weights are selected together with the stage coefficients to match the exact solution's Taylor expansion through fourth order. They are not based on a general claim that midpoint values are more representative. The resulting update is

$$u_{k+1} = u_k + \frac{\Delta t}{6}(y_1 + 2y_2 + 2y_3 + y_4).$$

To make the coefficient matching explicit, define the stage matrix B, weight vector b, and stage-time vector c:

$$B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 1/6 \\ 1/3 \\ 1/3 \\ 1/6 \end{bmatrix}, \quad c = B\mathbf{1} = \begin{bmatrix} 0 \\ 1/2 \\ 1/2 \\ 1 \end{bmatrix}.$$

Taylor-expanding each stage about the exact starting state and matching the independent derivative terms gives the following conditions through order four. This coefficient check supplements the RK4 formula in Lecture 10; c^j denotes componentwise powers and $C = \mathrm{diag}(c)$:

Order	Conditions verified by these coefficients
1	$b^T\mathbf{1} = 1$
2	$b^T c = 1/2$
3	$b^T c^2 = 1/3, \quad b^T B c = 1/6$
4	$b^T c^3 = 1/4, \quad b^T C B c = 1/8, \quad b^T B c^2 = 1/12, \quad b^T B^2 c = 1/24$

For example, $Bc = (0, 0, 1/4, 1/2)^T$, giving $b^T B c = 1/12 + 1/12 = 1/6$, while $B^2 c = (0, 0, 0, 1/4)^T$, giving $b^T B^2 c = 1/24$. These are the conditions arising from the nonlinear stage states, in addition to the quadrature moment conditions $b^T c^j = 1/(j+1)$. For explicit time dependence, the same argument applies after adding $t' = 1$ to the state. Under sufficient smoothness, the first unmatched terms are of order Δt^5. Appendix A shows the derivative terms that these conditions match.

2.2.3 Error

The classical fourth-order Runge–Kutta method matches the Taylor series expansion of the exact solution through terms of order Δt^4. Therefore, its **local truncation error**, defined as the error introduced by one step starting from the exact state, is fifth order:

$$E_{\text{local}} = O(\Delta t^5).$$

Lecture 11 divides this one-step defect by Δt, so its normalized truncation error is

$$\tau_k = E_{\text{local}}/\Delta t = O(\Delta t^4).$$

Over a fixed simulation interval, the number of steps is proportional to $1/\Delta t$. Provided the governing equations are sufficiently smooth and the integration remains stable, the accumulated **global truncation error** is fourth order:

$$E_{\text{global}} = O(\Delta t^4).$$

More explicitly, a Lipschitz bound on the one-step map gives $\|e_{k+1}\| \leq (1 + L\Delta t + O(\Delta t^2))\|e_k\| + C\Delta t^5$. Summing the resulting geometric bound over a fixed interval yields $\|e_N\| = O(\Delta t^4)$ for exact initial data. The constants depend on solution smoothness and the interval; the order is an asymptotic statement, not an error bound for an arbitrary step.

Consequently, halving the time step should reduce the global error by approximately a factor of $2^4 = 16$ when truncation error dominates.

The RK4 update approximates the integral of the governing differential equation over one time step:

$$u(t_{k+1}) = u(t_k) + \int_{t_k}^{t_{k+1}} f(u(t), t)\, dt.$$

Its weights,

$$\left(\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}\right),$$

mirror Simpson's Rule by emphasizing the midpoint of the interval. The two midpoint contributions have a combined weight of $4/6$, corresponding to Simpson's endpoint–midpoint–endpoint weights of $(1/6, 4/6, 1/6)$. However, RK4 evaluates its two midpoint slopes using different estimated states, so it is not generally identical to applying Simpson's Rule to the exact trajectory.

For the rocket model, the smoothness assumptions apply separately during powered ascent and coasting. Because thrust changes abruptly at burnout, each integration step must remain within a single flight phase. The solver ends a step exactly at burnout, uses the powered equations throughout that step, and then restarts with the coast equations. This avoids evaluating stages across the discontinuity and supports the expected fourth-order accuracy.

2.2.4 Stability

Numerical stability describes how perturbations, including roundoff and previously introduced errors, propagate through successive time steps. Applying RK4 to the linear test equation

$$\frac{du}{dt} = \lambda u$$

gives

$$u_{k+1} = R(z)u_k, \qquad z = \lambda\Delta t,$$

where the amplification factor is

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

For a strictly decaying test mode, the interior of the absolute-stability region satisfies

$$|R(\lambda\Delta t)| < 1.$$

The equality $|R| = 1$ describes the boundary and does not imply decay. Substituting $f = \lambda u$ into the stages gives $y_1 = \lambda u_k$, $y_2 = \lambda(1 + z/2)u_k$, $y_3 = \lambda(1 + z/2 + z^2/4)u_k$, and $y_4 = \lambda(1 + z + z^2/2 + z^3/4)u_k$; their weighted sum yields the stated $R(z)$, following Lecture 12.

RK4 has a bounded stability region and is therefore conditionally stable. An excessively large time step can cause numerical errors to grow even when the corresponding physical mode decays.

For the nonlinear rocket equations, local stability can be assessed using the eigenvalues of the Jacobian of the governing equations within each flight phase. For the equivalent two-state formulation with prescribed $m(t)$, during ascent ($v \geq 0$),

$$J = \begin{bmatrix} 0 & 1 \\ D/(mH) & -\rho C_d A v/m \end{bmatrix}.$$

For the implemented three-state formulation, the mass column is

$$[0, -(T-D)/m^2, 0]^T.$$

The third row is zero, introducing a zero eigenvalue. The two-state Jacobian has one positive and one negative eigenvalue when drag is nonzero and $v > 0$: growth in a physical perturbation is not automatically numerical instability. Decaying modes should lie within RK4's stability region after multiplication by Δt. Time-step refinement is also needed to establish that the predicted trajectory and apogee are sufficiently accurate, since numerical stability alone does not guarantee accuracy.

2.3 Algorithmic Summary

The logical execution flow of the custom RK4 simulation and apogee detection routine is structured into sequential steps, detailed below:

1. **Initialization:** Set the initial simulation time to zero, load the vehicle parameters, and define the starting state vector containing initial altitude, velocity, and mass.
2. **Main Time Loop:** Begin a loop that advances forward in time until the maximum simulation time is reached.
3. **Burn Status and Time-Step Adjustment:**
   - Check if the current time is within the motor's active burn duration.
   - Dynamically clip the time step size to ensure it does not exceed the remaining mission time or overshoot the exact instant of engine burnout.
4. **Fourth-Order Runge-Kutta (RK4) Evaluation:**
   - Compute the four intermediate slopes (k_1, k_2, k_3, k_4) by evaluating the governing equations of motion (incorporating atmospheric density, aerodynamic drag, thrust, and gravity) across the trial intervals.
   - Advance the flight state using the standard weighted average of the four slopes multiplied by the time step.
   - If engine burnout occurs within the current interval, explicitly enforce the final dry mass.
5. **Apogee Event Detection:** Monitor the vertical velocity across the step. If the velocity transitions from positive (climbing) to negative (falling), trigger a bisection root-finding routine to localize the numerical apogee, record the final apogee state, and terminate the simulation.
6. **Data Logging and Progression:** Record the updated time and state vector into the history logs, advance the current time clock, and proceed to the next iteration if apogee has not yet been reached.

3 Correct implementation

3.1 Core Solver Implementation

3.1.1 Explanation of Core Solver

The sounding rocket's equations of motion were implemented in Python using a fourth-order Runge-Kutta, RK4, numerical solver. The solver uses a state vector consisting of altitude, vertical velocity, and rocket mass.

This allows for all three variables to be advanced simultaneously at each time step. A derivative function was used to evaluate the effects of thrust, gravity, aerodynamic drag, and propellant consumption. The RK4 implementation calculates four intermediate estimates in order to determine the next state. To accurately represent the rocket's flight, the solver accounts for the transition between powered ascent and unpowered coasting. The time steps are adjusted to end exactly at the engine burnout. This ensures that the thrust and propellant mass loss are properly discontinued. The simulation continues until the vertical velocity reaches zero, which allows the maximum apogee altitude to be determined. The implementation was verified by comparing the numerical results with the analytical solution for the simplified flight conditions. Additional convergence studies were performed using progressively smaller time steps to evaluate numerical accuracy and confirm the expected fourth-order convergence behavior.

The complete source of `rocket_solver.py` is printed directly from the executable file in Appendix C; the experiment scripts are supplied in the repository.

The state is (h, v, m) throughout every RK4 stage. All stages of a step ending at burnout use the powered equations, including the endpoint stage, which evaluates the left-hand limit. The next step uses the coast equations. The state is continuous at burnout, and resetting m to the prescribed dry mass only removes floating-point drift from an exactly linear mass law.

When $v_k > 0$ and $v_{k+1} \leq 0$, 45 bisection iterations solve for a fractional RK4 step from the same left endpoint. This avoids identifying apogee with the largest sampled altitude. At a coasting apogee, $\dot{v} = -g \neq 0$, so the root is simple: an $O(\Delta t^4)$ state error produces an $O(\Delta t^4)$ event-time error locally, provided the crossing is bracketed and root-finding error is smaller. Numerical runs reject a failed lift-off, a nonfinite state, or reaching the time limit without locating apogee.

3.2 Convergence Rate Study

To verify the mathematical correctness and implementation order of the custom 4th-order Runge-Kutta (RK4) solver, a numerical convergence study was performed. The global discretization error was evaluated by comparing simulation outputs against a high-precision reference solution computed using an extremely refined time step ($\Delta t_{\text{ref}} = 0.0005\,\text{s}$).

The error E as a function of time step size Δt is modeled by the asymptotic power-law relation:

$$E(\Delta t) \approx C(\Delta t)^p \tag{2}$$

where p represents the empirical convergence rate (the slope on a log-log error plot). For a pure, unconstrained 4th-order Runge-Kutta method applied to a smooth differential equation, the theoretical convergence rate is $p = 4.0$.

3.2.1 Implementation

`convergence_study.py` evaluates a sequence of halved steps, compares each apogee with a refined numerical reference, computes adjacent-step orders, and fits the slope of $\log E$ against $\log \Delta t$. The independent analytical checks below are needed because self-convergence cannot detect a consistently incorrect differential equation.

3.2.2 Results and Discussion

The Table 1 convergence study uses $\Delta t = 1, 0.5, 0.25, 0.125$ s, with a reference step of 0.0005 s. The measured apogee errors are 7.87337×10^{-2}, 4.33276×10^{-3}, 2.53042×10^{-4}, and 1.52730×10^{-5} m. The adjacent-step observed orders,

$$p_{\text{obs}} = \log_2\left(\frac{E(\Delta t)}{E(\Delta t/2)}\right),$$

are 4.184, 4.098, and 4.050, respectively; the fitted slope is approximately 4.109. The trend toward four supports the expected RK4 order. Halving the reference step changes apogee by approximately 6.97×10^{-9} m, well below the smallest error fitted here.

A fit extending into the floating-point/reference-error floor can give a misleading slope. Burnout does not inherently reduce the order here because all stages remain within their selected flight phase. The 45 bisection iterations reduce the event-time bracket to at most $\Delta t/2^{45}$ in exact arithmetic; integration and floating-point

errors still remain. Convergence alone does not validate the governing equations, so independent checks follow.

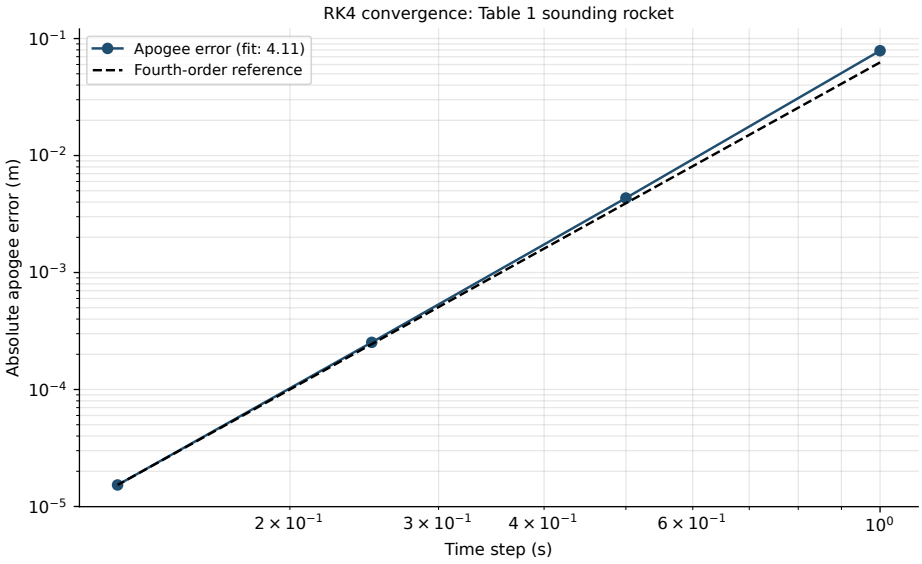


Figure 1: Apogee convergence for Table 1; adjacent-step orders approach four before the numerical error floor.

3.3 Independent Analytical Validation

While a standard error study (such as progressively halving the time step Δt) confirms that a numerical integration scheme converges at its theoretical rate, it is insufficient to prove the accuracy of the underlying physical model. A numerical solver will smoothly and predictably converge upon the solution of a fundamentally flawed differential equation—such as one with an inverted gravity vector or an incorrectly scaled mass flow rate. Therefore, to check that the solver evaluates the intended equations, its output is compared against a known, independent analytical solution.

3.3.1 Defining the Special Case Limit

The full Initial Value Problem (IVP) governing the ascent of the sounding rocket is defined by a complex sum of forces, including thrust (T), non-linear aerodynamic drag (D), and gravity (mg):

$$m\frac{dv}{dt} = T - \frac{1}{2}\rho v|v|C_d A - mg$$

Because air density (ρ) is a function of altitude and mass (m) varies with time during the engine burn, we use numerical integration for the full model. To create a mathematical benchmark, we isolate a special case limit where these complex terms drop out. By setting $C_d = 0$, $T = 0$, $t_{\text{burn}} = 0$, and $m_0 = m_{\text{dry}} = 50$ kg, the system is reduced to a constant-mass projectile operating in a vacuum. The governing equation simplifies dramatically as the mass term cancels out:

$$m\frac{dv}{dt} = -mg \implies \frac{dv}{dt} = -g$$

3.3.2 The Analytical Benchmark

By integrating this simplified linear limit with respect to time, the exact analytical solutions for velocity and altitude are derived as standard kinematic equations:

$$v(t) = \int -g\,dt = v_0 - gt$$

$$h(t) = \int (v_0 - gt)\, dt = h_0 + v_0 t - \frac{1}{2} g t^2$$

These exact equations serve as an absolute baseline. If the Python implementation of the physics engine and the RK4 loops are constructed correctly, the numerical output for this simplified state should agree with these kinematic solutions to floating-point accuracy.

3.3.3 Verification Execution and Analysis

The custom Runge-Kutta 4th Order (RK4) solver was configured with these simplified parameters and initialized with a starting velocity of $v_0 = 150$ m/s from an altitude of $h_0 = 0$ m. The simulation was executed with a time step of $\Delta t = 0.1$ seconds. At each discrete time step, the numerical altitude and velocity states were subtracted from the exact analytical states to calculate the absolute error.

The theoretical local truncation error of the RK4 method is $\mathcal{O}(\Delta t^5)$, for sufficiently smooth systems. For this constant-acceleration system, direct substitution into the RK4 stages gives $v_{k+1} = v_k - g\Delta t$ and $h_{k+1} = h_k + v_k \Delta t - g\Delta t^2/2$. Because the exact analytical solution for altitude under constant acceleration is a quadratic polynomial (degree 2), the mathematical truncation error for this specific test case is exactly zero. Upon running the simulation, the maximum recorded discrepancy between the RK4 numerical altitude and the exact analytical altitude was approximately 4.77×10^{-12} meters in the executed run. This small residual is consistent with accumulated floating-point roundoff; its absolute size depends on trajectory scale and the number of steps.

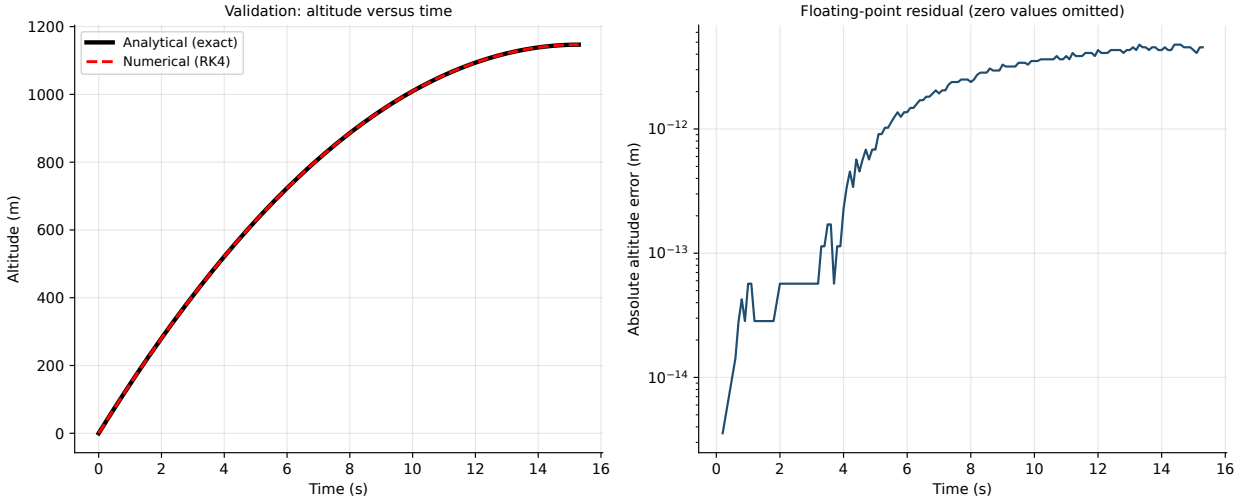


Figure 2: Ballistic vacuum validation with $v_0 = 150$ m/s and $\Delta t = 0.1$ s. Zero errors are omitted from the logarithmic error panel.

This test checks constant-acceleration kinematics, state indexing, and apogee localization, but does not test thrust or drag because they are disabled. It is evidence for this special case, not a proof that every part of the solver is correct.

An additional independently derived check retains thrust and changing mass while setting $C_d = 0$. During powered flight, let $q = \dot{m} > 0$ and $m(t) = m_0 - qt$. Integrating $\dot{v} = T_0/m(t) - g$ with $v(0) = h(0) = 0$ gives

$$v(t) = \frac{T_0}{q} \ln \frac{m_0}{m(t)} - gt, \qquad h(t) = \frac{T_0}{q^2} \left[m_0 - m(t) - m(t) \ln \frac{m_0}{m(t)} \right] - \frac{1}{2} g t^2.$$

For example, $\int_0^t \ln[m_0/(m_0 - qs)]\, ds = [m_0 - m(t) - m(t)\ln(m_0/m(t))]/q$, giving the altitude expression. Both expressions satisfy the initial conditions and differentiate back to the drag-free equations. At burnout, the altitude errors for $\Delta t = 0.5, 0.25, 0.125$ s are 3.5562×10^{-4}, 2.2283×10^{-5}, and 1.3936×10^{-6} m, approximately a factor of 16 per halving. This checks powered variable-mass integration. Separate automated checks verify drag direction, the exponential density ratio, non-grid-aligned burnout, and failure handling. No comparison with measured flight data has been performed.

3.4 Simulation Parameter Evaluation

The time step must be small enough to accurately predict apogee while remaining computationally efficient for repeated simulations of different burn durations and propellant mass fractions. This evaluation uses the physical parameters in Table 1, including $m_0 = 50\,\text{kg}$, $m_{\text{dry}} = 20\,\text{kg}$, $T_0 = 2500\,\text{N}$, and $t_{\text{burn}} = 10\,\text{s}$. The corresponding propellant mass flow rate is

$$\dot{m} = \frac{m_0 - m_{\text{dry}}}{t_{\text{burn}}} = 3\,\text{kg/s}.$$

For this evaluation, the numerical accuracy targets are chosen to be 0.1 m in apogee altitude and 0.001 s in apogee time, measured against a refined numerical reference.

3.4.1 Acceleration Variation and Flight-Phase Transitions

During ascent, the rocket's acceleration is

$$a = \frac{T - D}{m} - g, \qquad D = \frac{1}{2}\rho(h) C_d A v^2.$$

The rate of change of acceleration, or jerk, indicates how quickly the dynamics vary. Let $q = -dm/dt$, so that $q = \dot{m}$ during powered flight and $q = 0$ during coast. Within either smooth flight phase, differentiating the acceleration gives

$$\frac{da}{dt} = \frac{q(T - D)}{m^2} - \frac{\rho(h) C_d A}{m} va + \frac{Dv}{mH}.$$

These terms account for changing mass, velocity-dependent drag, and decreasing atmospheric density, respectively.

The refined nominal simulation gives a maximum sampled jerk magnitude of approximately $3.00\,\text{m/s}^3$ during powered ascent. Immediately after burnout, the jerk is approximately $28.95\,\text{m/s}^3$. Although the mass becomes constant during coast, the rocket initially experiences strong aerodynamic deceleration. Therefore, a larger coast time step cannot be justified solely by the absence of thrust; it must be supported by a numerical accuracy comparison.

Burnout also introduces an instantaneous acceleration change:

$$a(t_{\text{burn}}^+) - a(t_{\text{burn}}^-) = -\frac{T_0}{m_{\text{dry}}} = -125\,\text{m/s}^2.$$

For the nominal trajectory, acceleration changes from approximately $43.58\,\text{m/s}^2$ to $-81.42\,\text{m/s}^2$. This discontinuity is handled by shortening the final powered step to end exactly at t_{burn}. All four RK4 stages of that step use the powered equations, and the next step uses the coast equations. Consequently, no RK4 step mixes the two flight phases.

3.4.2 Time-Step Accuracy Study

The existing RK4 algorithm was evaluated with independently selected powered and coast time steps. All 25 combinations drawn from

$$\Delta t_{\text{powered}},\ \Delta t_{\text{coast}} \in \{0.1,\ 0.25,\ 0.5,\ 1.0,\ 2.0\}\,\text{s}$$

were tested. Each simulation retained exact burnout alignment and the existing 45-iteration bisection procedure for locating the numerical apogee event.

A uniform reference step of $\Delta t_{\text{ref}} = 0.0005\,\text{s}$ produced

$$h_{\text{ap,ref}} = 5806.866708\,\text{m}, \qquad t_{\text{ap,ref}} = 32.958766812\,\text{s}.$$

Halving the reference step to 0.00025 s changed these quantities by approximately 7.0×10^{-9} m and 7.9×10^{-11} s, respectively. These differences are negligible relative to the chosen accuracy targets. The reference is used to assess time-discretization error; it is not an independent validation of the physical model.

The errors are defined as

$$E_h = |h_{\text{ap}} - h_{\text{ap,ref}}|\,, \qquad E_t = |t_{\text{ap}} - t_{\text{ap,ref}}|\,.$$

Representative results are presented in Table 2, where N is the number of main RK4 steps, including the step that brackets apogee.

Table 2: Time-step accuracy and computational cost for the Table 1 rocket parameters.

$\Delta t_{\text{powered}}$ (s)	Δt_{coast} (s)	E_h (m)	E_t (s)	N
0.1	0.1	6.21×10^{-6}	8.07×10^{-9}	330
0.1	0.5	3.67×10^{-3}	4.02×10^{-6}	146
0.5	0.5	4.33×10^{-3}	6.03×10^{-6}	66
0.5	1.0	6.79×10^{-2}	8.51×10^{-5}	43
1.0	0.5	1.52×10^{-2}	3.95×10^{-5}	56
1.0	1.0	7.87×10^{-2}	1.19×10^{-4}	33
2.0	1.0	2.76×10^{-1}	7.44×10^{-4}	28
1.0	2.0	1.37	2.10×10^{-3}	22
2.0	2.0	1.57	2.73×10^{-3}	17

Increasing the coast step from 0.5 s to 1.0 s while retaining a powered step of 0.5 s reduces the main step count from 66 to 43 while satisfying both accuracy targets. Increasing the powered step to 1.0 s further reduces the count to 33. However, assigning 2.0 s to either phase fails the altitude tolerance for every tested combination.

A local stability check using the Jacobian of the governing equations gives a most negative eigenvalue of approximately $-0.333\,\text{s}^{-1}$ along the nominal trajectory. For $\Delta t = 1.0\,\text{s}$, the RK4 amplification factor for this fastest decaying mode is approximately 0.717, within the method's absolute stability region. This local check supports the selected step, while the refinement results establish its accuracy for the quantities of interest.

3.4.3 Selected Parameters and Application to the Results

For the nominal Table 1 case, the selected setting is

$$\boxed{\Delta t_{\text{powered}} = \Delta t_{\text{coast}} = 1.0\,\text{s}}$$

because it is the least expensive of the 25 tested combinations that satisfies both prescribed tolerances. It predicts

$$h_{\text{ap}} = 5806.787974\,\text{m}, \qquad t_{\text{ap}} = 32.958648254\,\text{s}.$$

Including apogee localization, this setting requires 316 derivative evaluations, compared with 1504 for a uniform 0.1 s step, a reduction of approximately 79%. If greater numerical margin is desired, a uniform 0.5 s step reduces the apogee error to approximately 0.00433 m.

The maximum simulation time remains 120 s, which exceeds the nominal apogee time. Integration terminates at the localized zero-velocity event rather than selecting the largest sampled altitude. Any simulation that reaches the time limit before this event raises an error and must be extended rather than treating its final altitude as apogee. Configurations that cannot lift off immediately are also rejected.

The 1 s choice is specific to the nominal case. The Results study instead uses uniform 0.125 s steps, with 0.5 and 0.25 s comparisons for every configuration. Both successive differences satisfy the same 0.1 m and 0.001 s criteria. The largest final-halving differences are 0.00264 m and 1.63×10^{-5} s; these are observed refinement differences, not certified exact-error bounds. A configuration-dependent time limit of $\max(120\,\text{s}, 2t_{\text{ap,vac}})$ permits the longer vacuum flights. All runs terminate at the apogee event. Detailed study controls and numerical safeguards are given in Section 4.

4 Results

4.1 Study Design and Nominal Flight

To study the effects of burn duration and propellant mass fraction, we used 21 burn durations (5–15 s in 0.5 s increments) and 17 propellant fractions (0.40–0.80 in increments of 0.025), for 357 configurations. This range brackets the nominal (10 s, 0.60) case and probes both parameters without approaching the lift-off boundary. Thrust is fixed at 2500 N, dry mass at 20 kg, and all geometry and atmospheric quantities at Table 1 values. The initial mass consequently ranges from 33.33 to 100 kg; even the heaviest case has $T_0/(m_0 g) = 2.55 > 1$.

Each atmospheric trajectory is integrated at $\Delta t = 0.5$, 0.25, and 0.125 s. The finest trajectory supplies the reported results, while both successive refinements are checked against the selected altitude and time tolerances. Drag-free trajectories at 0.125 s are independently compared with the exact powered-plus-coasting solution in Appendix A.2. Across all 357 cases, the largest vacuum discrepancies are 6.27×10^{-4} m in apogee and 3.30×10^{-6} s in apogee time. The drag penalty uses the analytical vacuum apogee to avoid an additional numerical-reference error:

$$\Delta h_D = h_{\mathrm{ap,vac}} - h_{\mathrm{ap,atm}}, \qquad P_D = 100 \frac{\Delta h_D}{h_{\mathrm{ap,vac}}} \quad (\%). \tag{3}$$

This difference includes drag's effects during powered flight and coast. It is an altitude penalty between two complete trajectories, not a direct drag-work measurement.

For the nominal case, atmospheric apogee is 5806.87 m at 32.95877 s. Figure 3 provides the trajectory context; the parameter comparisons that answer the engineering question follow in Figures 4–6. Mass decreases linearly to 20 kg at 10 s, remains constant thereafter, and upward velocity decreases to zero at apogee. The state is continuous at burnout even though acceleration jumps.

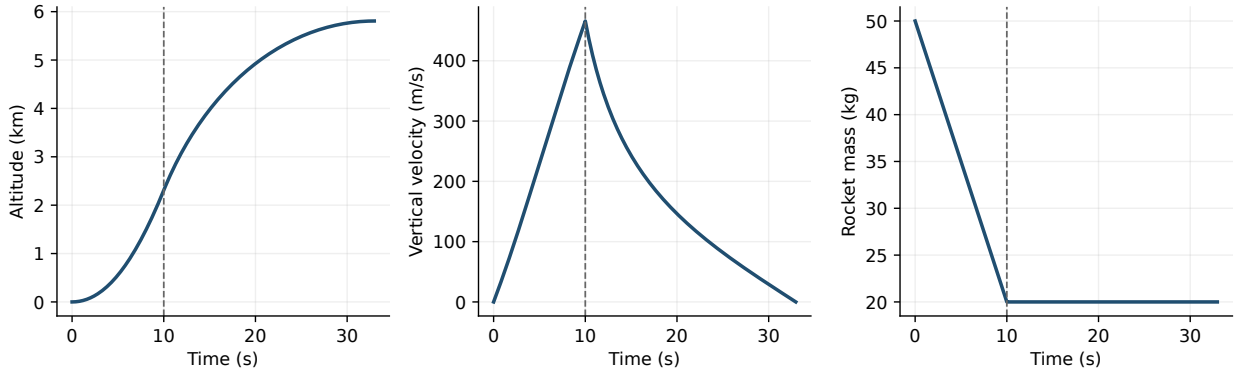


Figure 3: Nominal atmospheric trajectory ($\phi = 0.60$, $t_{\mathrm{burn}} = 10$ s), plotted at $\Delta t = 0.1$ s. Dashed lines mark burnout. The final point is the localized zero-velocity event.

4.2 Effect of Motor Burn Duration

At every sampled propellant fraction, atmospheric apogee increases with burn duration. For the nominal fraction $\phi = 0.60$, increasing the duration from 5 to 10 to 15 s gives 2613.40, 5806.87, and 9669.68 m, respectively. The 15 s case reaches approximately 3.70 times the altitude of the 5 s case. The smallest altitude gain between adjacent 0.5 s duration samples anywhere on the grid is 209.80 m, far larger than the observed refinement changes.

This trend follows from the specified comparison: at fixed thrust, impulse increases from 12,500 to 37,500 N s as the burn is lengthened. At fixed propellant fraction the propellant mass stays constant, so a longer burn also lowers the mass-flow rate and increases the implied effective exhaust velocity $T_0/\dot{m}$. The result therefore does not isolate a burn-time effect at fixed impulse or fixed exhaust velocity.

The exact constant-gravity vacuum solution has $h_{\mathrm{ap,vac}} \propto t_{\mathrm{burn}}^2$ at a fixed propellant fraction (Appendix A.2). The atmospheric case does not follow that simple scaling because drag couples altitude, density,

and velocity. Near the nominal case, a centered difference using the 9.5 and 10.5 s runs gives

$$\left.\frac{\partial h_{\mathrm{ap}}}{\partial t_{\mathrm{burn}}}\right|_{\phi=0.6,t_{\mathrm{burn}}=10\,\mathrm{s}} \approx 686.89\ \mathrm{m/s}.$$

This is a finite-difference sensitivity over the stated spacing, not a universally constant slope.

4.3 Effect of Propellant Mass Fraction

At every sampled burn duration, atmospheric apogee decreases as propellant fraction increases. For $t_{\mathrm{burn}} = 10$ s, raising ϕ from 0.40 to 0.60 to 0.80 changes apogee from 7075.29 to 5806.87 to 3677.64 m. The endpoints differ by 3397.65 m, a decrease of approximately 48.0%. The smallest decrease between adjacent fraction samples anywhere on the grid is 74.39 m, again well above the measured numerical changes.

With dry mass and burn duration fixed, additional propellant increases the initial mass and the mass throughout the powered interval, while total impulse is unchanged. In this parameterization,

$$m(t;\phi) = m_{\mathrm{dry}} + \frac{m_{\mathrm{dry}}\phi}{1-\phi}\left(1 - \frac{t}{t_{\mathrm{burn}}}\right), \qquad 0 \le t \le t_{\mathrm{burn}}.$$

Increasing ϕ therefore reduces thrust acceleration T_0/m before burnout. It also reduces the implied exhaust velocity. These changes explain why adding propellant does not improve performance in this fixed-impulse comparison. The measured trend is not a general claim that more propellant worsens a rocket with a fixed-specific-impulse motor.

A centered difference at the nominal point, using $\phi = 0.575$ and 0.625, gives $\partial h_{\mathrm{ap}}/\partial\phi \approx -7891.55$ m per unit fraction. Locally, a 0.01 increase in fraction corresponds to approximately 78.9 m less altitude, with other prescribed quantities held fixed. This local estimate should not be extrapolated over the entire parameter range.

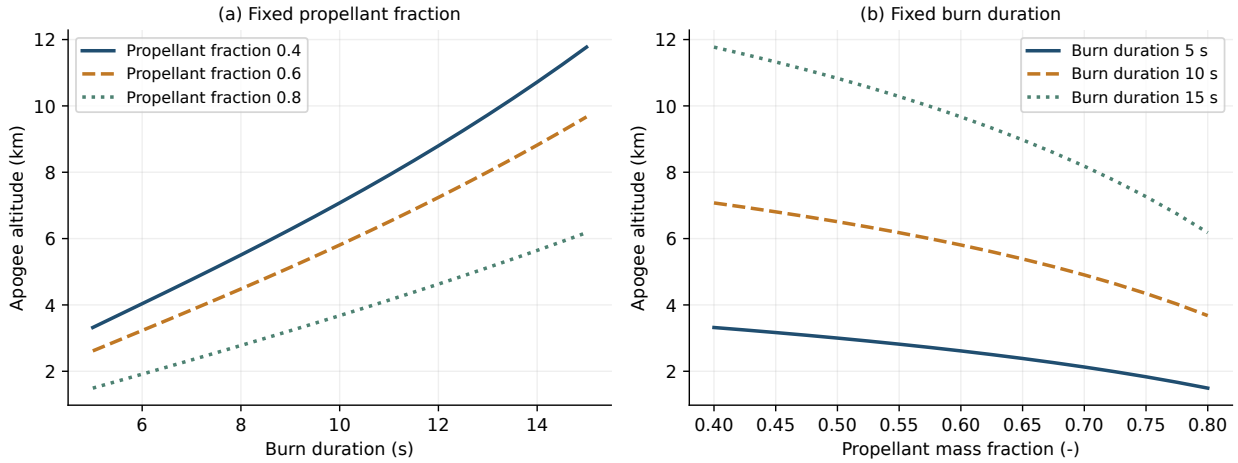


Figure 4: Controlled atmospheric sensitivity comparisons. (a) Burn-duration sweeps at three fixed propellant fractions. (b) Fraction sweeps at three fixed burn durations. All curves use $\Delta t = 0.125$ s, fixed thrust and fixed dry mass; line styles distinguish cases in grayscale.

4.4 Combined Effects and Atmospheric Drag

Figure 5 shows the joint dependence. The largest sampled atmospheric apogee is 11771.92 m at $(15\,\mathrm{s}, 0.40)$; the smallest is 1491.70 m at $(5\,\mathrm{s}, 0.80)$. Both occur on the study boundary. There is no interior maximum among the sampled grid points. This is a statement about the selected grid and constraints, not a proof of a global continuous optimum or a recommendation for a realizable motor.

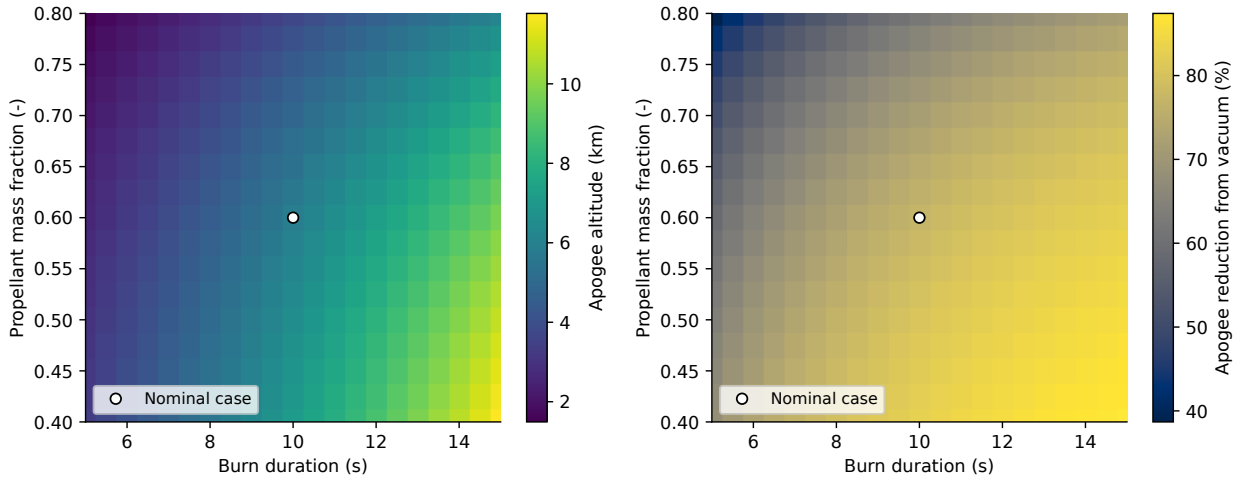


Figure 5: The 357-case design grid. Left: atmospheric apogee. Right: relative altitude penalty from Eq. (3). White circles locate the nominal case. Colored cells represent computed samples, not additional simulations between samples.

For the nominal configuration, removing drag raises the modeled apogee from 5806.87 to 25324.08 m, giving $\Delta h_D = 19517.22$ m and $P_D = 77.07\%$. Over the full grid, P_D ranges from 38.68% to 87.41%. The relative drag penalty increases with burn duration at each sampled fraction and decreases with fraction at each sampled burn duration. Higher-performing vacuum configurations lose a larger fraction of their potential altitude to drag in this model.

The strong loss is consistent with quadratic aerodynamic resistance during high-speed ascent. The exponential atmosphere weakens drag at higher altitude, but does not eliminate its accumulated effect. Figure 6 compares the two models directly, while Table 3 gives representative values. Absolute penalties and all other grid values are available in `data/engineering_results.csv`.

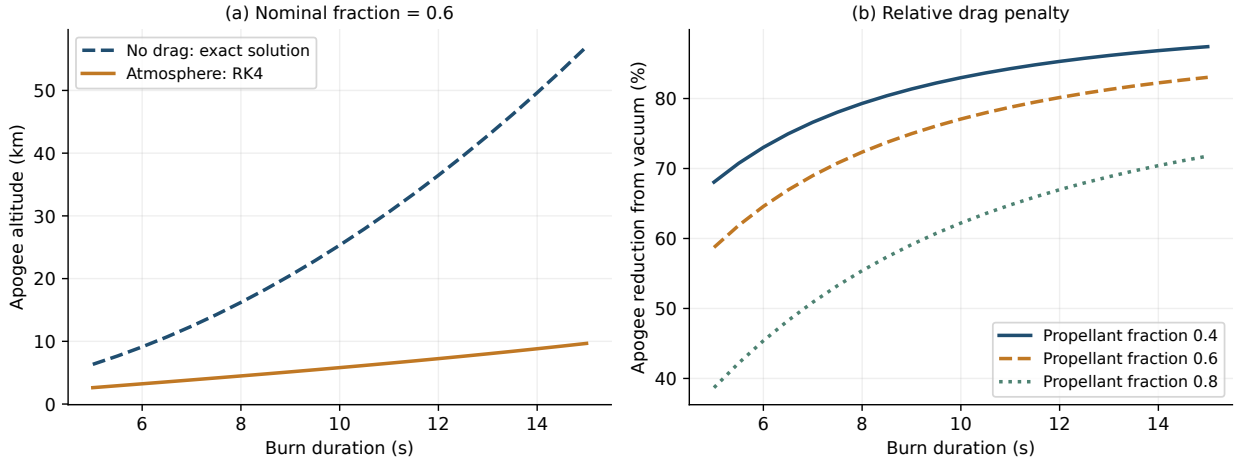


Figure 6: Atmospheric drag penalty. (a) Exact vacuum and numerical atmospheric apogees at $\phi = 0.60$. (b) Relative reduction from the vacuum apogee for three fractions. Both models use the same constant gravity and vehicle parameters.

4.5 Engineering Interpretation and Model Limits

Within the stated family, longer burns and lower propellant fractions produce greater apogee, while drag removes a substantial fraction of the corresponding vacuum altitude. The two design variables cannot be

Table 3: Representative configurations from the executed grid. Atmospheric quantities use $\Delta t = 0.125$ s; vacuum apogees are analytical. Values are rounded for presentation.

t_{burn} (s)	ϕ	$h_{\text{ap,atm}}$ (m)	$t_{\text{ap,atm}}$ (s)	$h_{\text{ap,vac}}$ (m)	P_D (%)
5	0.40	3319.15	24.470	10390.57	68.06
10	0.40	7075.29	34.720	41562.29	82.98
15	0.40	11771.92	45.375	93515.16	87.41
5	0.60	2613.40	22.825	6331.02	58.72
10	0.60	5806.87	32.959	25324.08	77.07
15	0.60	9669.68	42.689	56979.18	83.03
5	0.80	1491.70	19.239	2432.75	38.68
10	0.80	3677.64	29.467	9731.00	62.21
15	0.80	6179.63	38.044	21894.75	71.78

interpreted independently of the constraints: the implied exhaust velocity varies from 156.25 to 2812.50 m/s across the grid. A practical motor comparison would need a specified impulse or exhaust-velocity constraint and a compatible thrust/mass-flow law. Those are different studies from the fixed-thrust question answered here.

The parameters are illustrative rather than fitted to a specific vehicle. Constant C_d neglects Mach and Reynolds-number dependence; vertical motion neglects wind, attitude, and lift; constant g and an exponential atmosphere simplify the environment. The largest vacuum apogee is 93.52 km, where constant gravity is particularly limiting. Vacuum trajectories here are controlled mathematical comparisons, not realistic high-altitude flight forecasts. Numerical convergence and exact-solution tests establish evidence for solving the specified equations; they do not quantify uncertainty in these physical assumptions. No experimental validation or global motor optimization is claimed.

5 Contributions

The group worked collaboratively, with each member contributing approximately 25% of the original project work. The roles reported by the group are summarized below. Additional AI assistance with the calculations, Results section, and revisions is described in the Acknowledgments.

- **Maria Dalton (25%):** Maria explained how the rocket's motion could be formulated as an initial value problem, connecting the known launch conditions to the equations for altitude, velocity, and mass. She also developed the justification for choosing RK4 by discussing its accuracy, stability, and computational cost. For the implementation, Maria worked on the initial Python solver and the logic for switching from powered flight to coasting at burnout. Her work connected the mathematical model to the numerical time-stepping procedure used to calculate the trajectory and apogee.

- **Morgan Waller (25%):** Morgan helped frame the main engineering question about how burn duration and propellant mass fraction affect apogee. For the method section, Morgan developed the discussion of RK4's error and stability properties, including the distinction between the one-step error and the accumulated global error. Morgan also worked on the simulation-parameter evaluation, comparing time-step choices in the powered and coast phases. This work focused on balancing numerical accuracy with computational cost and explaining why a stable time step must also be checked for accuracy.

- **Yogeshwari Kant (25%):** Yogeshwari defined the mathematical functions and parameters used in the rocket model, including the thrust profile, mass depletion, atmospheric density, and aerodynamic drag. She also wrote the algorithmic summary describing how the solver advances the state, handles burnout, and detects apogee. For the convergence study, Yogeshwari worked on the time-step refinement procedure and the analysis of the resulting errors. Her contribution helped explain how a log-log error plot and observed convergence orders can be used to compare the implementation with RK4's expected fourth-order behavior.

- **Justin Chiang (25%):** Justin explained why numerical integration is useful for the coupled, nonlinear rocket equations. He developed the Taylor-series foundation of RK4 and the explanations of its four intermediate slopes and weighted update. For the independent validation, Justin worked on the constant-mass, zero-drag ballistic benchmark and the comparison between numerical trajectories and the exact kinematic solution. This provided a check beyond convergence against a refined numerical result. Justin also prepared the original reproducibility instructions, contribution descriptions, and AI acknowledgment, documenting how the work was organized and how the calculations could be repeated.

6 Reproducibility

Code repository:
`https://github.com/justinchiang0207/AE370-Sounding-Rocket-Ascent---Initial-Value-Problem`

The package separates the numerical solver, verification experiments, engineering parameter study, and plotting style. Install the pinned dependencies in `requirements.txt`, then run `python reproduce.py`. The command executes seven automated tests, all five experiment scripts, and PDF compilation when a local LaTeX installation is available. `python reproduce.py --skip-latex` regenerates all numerical results without a TeX installation. All outputs are written relative to the script directory; no external time-integration solver is used.

The `data/` directory contains raw trajectory and convergence CSV files, all 25 step-pair comparisons, all 357 engineering cases, diagnostic JSON, the generated Results table, an environment record, and an execution log. Figures are exported in vector PDF and raster PNG formats. Appendix B maps scripts to outputs. Numerical differences at the last few floating-point digits may occur across platforms.

For Overleaf, upload the entire project ZIP and select `main.tex` with pdfLaTeX. The report reads its solver listing from `rocket_solver.py` and its Results table from `data/results_table.tex`; these files must accompany the figures. Python is not executed by Overleaf. The README gives GitHub upload and local reproduction instructions.

7 Acknowledgments

We first brainstormed our project idea as a group and then used ChatGPT to check whether the idea and its assumptions were reasonable. After writing an initial draft, we asked AI to review it and suggest revisions. We followed a similar process for the code: we first attempted to write it ourselves, then used AI to review the implementation, identify errors, and suggest adjustments. We also asked AI for explanations and guidance when we were stuck or unsure how to proceed.

AI assistance included developing additional verification checks, running the parameter studies, generating figures, and helping interpret the results. Finally, we gave the full document to AI for an overall review, including checking technical consistency, organizing the formatting, correcting grammar, and rephrasing unclear sentences. We reviewed the suggested changes and checked the revised equations, code, and results before including them in the report.

A Supporting Derivations

A.1 Nonlinear Taylor Matching for RK4

Within one phase the three-state model is autonomous: $u' = F(u)$. Let $J = F'(u)$, F'' denote its bilinear second derivative, and F''' its trilinear third derivative, all evaluated at the exact initial state. Successive differentiation gives

$$\begin{aligned} u' &= F, \qquad u'' = JF, \\ u''' &= F''(F,F) + J^2F, \\ u^{(4)} &= F'''(F,F,F) + 3F''(F,JF) + JF''(F,F) + J^3F. \end{aligned}$$

The exact expansion is therefore

$$\begin{aligned} u(t+\Delta t) = u + \Delta t F + \frac{\Delta t^2}{2} JF + \frac{\Delta t^3}{6}\{F''(F,F) + J^2F\} \\ + \frac{\Delta t^4}{24}\{F'''(F,F,F) + 3F''(F,JF) + JF''(F,F) + J^3F\} + O(\Delta t^5). \end{aligned}$$

Expanding each stage $F(u+\delta) = F + J\delta + F''(\delta,\delta)/2 + F'''(\delta,\delta,\delta)/6 + \cdots$ and substituting recursively gives the RK coefficients for these terms:

Derivative term	RK coefficient	Exact coefficient
F	$b^T\mathbf{1}$	1
JF	b^Tc	1/2
$F''(F,F)$	$b^Tc^2/2$	1/6
J^2F	b^TBc	1/6
$F'''(F,F,F)$	$b^Tc^3/6$	1/24
$F''(F,JF)$	b^TCBc	1/8
$JF''(F,F)$	$b^TBc^2/2$	1/24
J^3F	b^TB^2c	1/24

Here B, b, c, C are those in Section 2. Inserting the classical RK4 coefficients satisfies each equality and establishes fourth-order matching for nonlinear systems, not merely the linear test equation. This is an independent expansion supplementing the lecture presentation. The derivation applies separately on each side of burnout; it does not assume a differentiable thrust function across the switch.

A.2 Exact Powered-Plus-Coast Vacuum Apogee

Let $q = (m_0 - m_{\text{dry}})/t_{\text{burn}} > 0$ and set $C_d = 0$. With $h(0) = v(0) = 0$, direct integration gives

$$\begin{aligned} v(t) &= \frac{T_0}{q}\ln\frac{m_0}{m_0 - qt} - gt, \\ h(t) &= \frac{T_0}{q^2}\left[qt - (m_0 - qt)\ln\frac{m_0}{m_0 - qt}\right] - \frac{1}{2}gt^2. \end{aligned}$$

At $t = t_{\text{burn}}$, denote these values by h_b, v_b. In coast, for $s = t - t_{\text{burn}}$, $v = v_b - gs$ and $h = h_b + v_b s - gs^2/2$. If $v_b > 0$ and $g > 0$, then

$$t_{\text{ap,vac}} = t_{\text{burn}} + \frac{v_b}{g}, \qquad h_{\text{ap,vac}} = h_b + \frac{v_b^2}{2g}.$$

All study configurations lift off immediately and have $v_b > 0$. At fixed ϕ and dry mass, let $m_p = m_0 - m_{\text{dry}}$ and $L = \ln(m_0/m_{\text{dry}})$, both independent of burn duration. Substituting $q = m_p/t_{\text{burn}}$ gives

$$h_{\text{ap,vac}} = t_{\text{burn}}^2\left\{\frac{T_0}{m_p^2}(m_p - m_{\text{dry}}L) - \frac{g}{2} + \frac{1}{2g}\left(\frac{T_0L}{m_p} - g\right)^2\right\}.$$

Thus the vacuum altitude scales quadratically with duration under these particular constraints. The bracket has units of acceleration, so multiplication by time squared gives altitude. This formula is used as an independent check of the numerical solver, including the powered/coast transition and apogee localization.

A.3 Signs and Dimensions

The positive axis points upward. Thrust is positive during burn; gravity contributes $-g$; and $-\rho C_d A v|v|/2$ opposes either direction of motion. Density has units kg/m^3, so $\rho A v|v|$ has units N and D/m has units m/s^2. The exponent h/H and mass ratio inside the logarithm are dimensionless. The mass-flow rate q has units kg/s, T_0/q has units m/s, and all analytical solutions satisfy $h(0) = v(0) = 0$ and $m(0) = m_0$.

B Reproducibility File Map

File	Purpose and principal outputs
`rocket_solver.py`	Parameters, signed-drag derivative, RK4 stages, burnout alignment and apogee event.
`plot_trajectory.py`	Nominal trajectory CSV; altitude, velocity, mass and combined trajectory figures.
`convergence_study.py`	Reference refinement, apogee-error CSV, adjacent orders, fitted convergence figure.
`validation.py`	Independent ballistic and powered vacuum solutions; validation CSV files and figure.
`parameter_evaluation.py`	All 25 phase-step pairs, RHS cost counts and nominal diagnostics.
`engineering_results.py`	All 357 configurations, three-level refinement, exact-vacuum comparisons, three engineering figures, summary JSON and report table.
`plot_style.py`	Common plotting appearance; vector PDF and raster PNG export.
`test_solver.py`	Seven tests: ballistic limit, powered order, nonaligned burnout, drag coefficient/sign/density, rejected failures, vacuum apogee, and evaluation count.
`reproduce.py`	Runs tests and experiments; records environment and execution log; compiles report when available.
`requirements.txt`	Pinned NumPy and Matplotlib versions used for the executed study.
`README.md`	Setup, model assumptions, commands, expected outputs, and GitHub/Overleaf instructions.
`data/`, `figures/`	Generated numerical evidence and figures referenced in the report.

C Complete Core Solver

The listing below is read directly from the executed source file, preventing a separately maintained code transcription. The experiment and test scripts accompany it in the repository.

C.1 Parameters, Derivatives, and RK4 Step

Listing 1: Core solver: physical parameters, derivative evaluation, and RK4.

```python
from dataclasses import dataclass
from math import exp, isfinite
from typing import Tuple

State = Tuple[float, float, float]

@dataclass(frozen=True)
class RocketParameters:
    initial_mass: float = 50.0       # kg, including 30 kg propellant
    dry_mass: float = 20.0           # kg, after burnout
    burn_time: float = 10.0          # s
    thrust: float = 2500.0           # N during burn
    drag_coefficient: float = 0.45    # assumed constant
    area: float = 0.0314             # m^2, approximately 20 cm diameter
    sea_level_density: float = 1.225 # kg/m^3
    scale_height: float = 8500.0     # m
    gravity: float = 9.81            # m/s^2

    def __post_init__(self):
        if not all(isfinite(x) for x in vars(self).values()):
            raise ValueError("Parameters must be finite")
        if not (0 < self.dry_mass <=  self.initial_mass):
            raise ValueError("Require 0 < dry_mass <= initial_mass")
        if self.burn_time < 0 or (self.burn_time == 0 and
                                  self.initial_mass != self.dry_mass):
            raise ValueError("Positive propellant mass requires burn_time > 0")
        if (self.thrust < 0 or self.drag_coefficient < 0 or self.area < 0
                or self.sea_level_density < 0 or self.gravity < 0
                or self.scale_height <= 0):
            raise ValueError("Check the physical parameter signs")

    @property
    def mass_flow_rate(self) -> float:
        if self.burn_time == 0:
            return 0.0
        return (self.initial_mass - self.dry_mass) / self.burn_time


def derivatives(state: State, p: RocketParameters, burning: bool) -> State:
    height, velocity, mass = state
    if mass <= 0:
        raise ValueError("Rocket mass must remain positive")

    density = p.sea_level_density * exp(-height / p.scale_height)
    drag = 0.5 * density * p.drag_coefficient * p.area * velocity * abs(velocity)
    thrust = p.thrust if burning else 0.0
    mass_derivative = -p.mass_flow_rate if burning else 0.0

    height_derivative = velocity
    velocity_derivative = (thrust - drag) / mass - p.gravity
    return height_derivative, velocity_derivative, mass_derivative

def add_scaled(state: State, slope: State, factor: float) -> State:
    return tuple(x + factor * k for x, k in zip(state, slope))

def rk4_step(state: State, dt: float, p: RocketParameters,
             burning: bool) -> State:
    k1 = derivatives(state, p, burning)
    k2 = derivatives(add_scaled(state, k1, dt / 2), p, burning)
    k3 = derivatives(add_scaled(state, k2, dt / 2), p, burning)
    k4 = derivatives(add_scaled(state, k3, dt), p, burning)
    return tuple(
        x + dt * (a + 2*b + 2*c + d) / 6
        for x, a, b, c, d in zip(state, k1, k2, k3, k4)
    )
```

C.2 Apogee Localization and Time Marching

```
def apogee_within_step(state: State, step: float,
                       p: RocketParameters, burning: bool):
    low, high = 0.0, step
    for _ in range(45):
        middle = 0.5 * (low + high)
        trial = rk4_step(state, middle, p, burning)
        if trial[1] > 0:
            low = middle
        else:
            high = middle
    offset = 0.5 * (low + high)
    return offset, rk4_step(state, offset, p, burning)

def simulate(p: RocketParameters, dt: float = 0.1, end_time: float = 120.0,
             initial_height: float = 0.0, initial_velocity: float = 0.0,
             stop_at_apogee: bool = True, dt_coast: float = None):
    """Stop at the first upward-to-downward crossing; raise if not reached.

    A launch from rest requires positive initial upward acceleration.
    Ground contact, delayed lift-off, and relaunch are outside this model.
    Set stop_at_apogee=False only for fixed-time verification studies.
    """
    if dt_coast is None:
        dt_coast = dt
    if not all(isfinite(x) for x in
               (dt, dt_coast, end_time, initial_height, initial_velocity)):
        raise ValueError("Numerical inputs must be finite")
    if dt_coast <= 0:
        raise ValueError("dt_coast must be positive")
    if dt <= 0 or end_time <= 0:
        raise ValueError("dt and end_time must be positive")

    t = 0.0
    state: State = (initial_height, initial_velocity, p.initial_mass)
    if stop_at_apogee and (initial_velocity < 0 or
            (initial_velocity == 0 and
             (p.thrust if p.burn_time > 0 else 0.0) / p.initial_mass <= p.gravity)):
        raise ValueError("Apogee run requires upward flight or immediate lift-off")
    times = [t]
    states = [state]

    while t < end_time:
        burning = t < p.burn_time
        step = min(dt if burning else dt_coast, end_time - t)

        if burning:
            step = min(step, p.burn_time - t)

        next_t = t + step
        if burning and abs(next_t - p.burn_time) <= 1e-12:
            next_t = p.burn_time
            step = next_t - t
        if step <= 0:
            raise ArithmeticError("Time step cannot advance time")

        next_state = rk4_step(state, step, p, burning)
        if not all(isfinite(x) for x in next_state):
            raise ArithmeticError("Nonfinite state; reduce the time step")
        if burning and next_t == p.burn_time:
            next_state = (next_state[0], next_state[1], p.dry_mass)

        if stop_at_apogee and state[1] > 0 and next_state[1] <= 0:
            offset, peak_state = apogee_within_step(state, step, p, burning)
            times.append(t + offset)
            states.append(peak_state)
            return times, states

        t, state = next_t, next_state
        times.append(t)
        states.append(state)

    if stop_at_apogee:
        raise RuntimeError("Apogee not reached; increase end_time")
    return times, states
```